

3.2 Setting the Parameter for Stochastic Perturbation

The parameter σ specifies the range of stochastic perturbation. In the following, we will consider two cases of setting σ , fixed and trainable.

3.2.1 Fixed Case

There are several ways to choose the desired σ . The simplest is setting σ to be a constant hyper-parameter. Choosing one constant σ for all elements is theoretically justified as ϵ is randomly sampled from a normal distribution and σ acts as a scaling factor to the sampled value ϵ . This can be interpreted as repeated addition of the scaled Gaussian noise to the activation maps, which helps in better convergence of the network parameters [24]. The network is optimized using gradient-based learning.

The nature of $z = \sigma\epsilon$ is Gaussian as $\epsilon \sim \mathcal{N}(0, 1)$, and σ being a constant value which does not affect the Gaussian properties. This ensures learning using gradient-based methods. The proposed method does not significantly affect the number of parameters in the architecture, hence comes at no additional computational cost. However, choosing the best σ is a difficult task as setting any other hyper-parameter for training a neural network.

3.2.2 Trainable Case

Using a trainable σ reduces the requirement to determine σ as a hyper-parameter and allows the network to learn the appropriate range of sampling. There are two ways of introducing a trainable σ :

- *Single Trainable σ* : A shared trainable σ across the network. This introduces a single extra parameter used for all ProbAct layers. This is similar to the fixed σ but the value is trained.
- *Element-wise Trainable σ* : This method uses a trainable parameter for each input element. This adds the flexibility to learn a different distribution for every input-output mapping.

Training σ The trainable parameter σ is trained using a back-propagation simultaneously with other model parameters. The gradient computation of σ is done using the chain rule. Given an objective function $\mathcal{E}$, the gradient of $\mathcal{E}$ with respect to $\sigma_{l,i}$, where $\sigma_{l,i}$ is the perturbation parameter and $y_{l,i}$ is the output of the i -th unit in the l -th layer, is:

$$\frac{\partial \mathcal{E}}{\partial \sigma_{l,i}} = \frac{\partial \mathcal{E}}{\partial y_{l,i}} \frac{\partial y_{l,i}}{\partial \sigma_{l,i}}, \quad (4)$$

The term $\frac{\partial \mathcal{E}}{\partial y_{l,i}}$ is the gradient propagated from the deeper layer. The gradient of the activation is given by:

$$\frac{\partial y_{l,i}}{\partial \sigma_{l,i}} = \epsilon. \quad (5)$$

Bounded σ Training σ without any bounds can create perturbations in a highly unpredictable manner when $\sigma \rightarrow \infty$, making training difficult. Taking the advantages of monotonic nature of the sigmoid function, we bound the upper and lower limit of σ to $(0, \alpha)$ using:

$$\sigma = \alpha \text{sigmoid}(\beta k), \quad (6)$$

where k represents the element-wise learnable parameter, and α and β are scaling parameters that can be set as hyper-parameters.

3.3 Stochastic Regularizer

Figure 1 (c) illustrates the effect of stochastic perturbation by ProbAct in the feature space of each neural network layer. Intuitively, ProbAct adds perturbation to each feature vector independently, and this function acts as a regularizer to the network. It should be noted that while the noise added to each ProbAct is isotropic, the noise from early layers is propagated to the subsequent layers; hence, the total noise added to a certain layer depends on the noise and weights of the early layers.

Further, the effect of regularization is proportional to the variance of the distribution. A high variance is induced with a higher σ value, allowing sampling from a high variance distribution which is further

Table 1: Performance comparison of various activation functions and a Variation Inference Neural Network with ProbAct. The test accuracy(%) indicates the average of testing over three runs. The train and test time is reported with respect to ReLU activation function and is measured in seconds and milliseconds respectively.

Activation function	CIFAR-10	CIFAR-100	STL-10	IMDB	Train time (sec)	Test time (milli-sec)
Sigmoid	10.00	1.00	10.00	85.92	1.07	1.03
Tanh	10.00	1.00	10.00	85.88	1.08	1.03
ReLU	87.27	52.94	60.80	85.85	1.00	1.00
Leaky ReLU	86.49	49.44	59.16	85.47	1.04	1.08
PReLU	86.35	46.30	60.01	85.95	1.16	1.00
ELU	87.65	56.60	64.11	86.51	1.16	1.04
SELU	86.65	51.52	60.71	85.71	1.19	1.05
Swish	86.55	54.01	63.50	86.14	1.20	1.13
Bayesian VGG VI ²	86.22	48.27	57.22	-	-	-
ProbAct						
Mean						
Element-wise μ	85.80	48.50	54.17	83.86	1.29	1.35
Sigma						
Fixed ($\sigma = 0.5$)	88.50	56.85	62.30	87.31	1.09	1.25
Fixed ($\sigma = 1.0$)	88.87	58.45	62.50	87.00	1.10	1.27
One Trainable σ	87.40	53.87	63.07	86.35	1.23	1.30
EW Trainable σ						
Unbound	86.40	54.10	61.70	86.64	1.25	1.31
Bound	88.92	55.83	64.17	85.86	1.26	1.33

away from the mean. This way the prediction is not over-reliant on one value, helpful in countering overfitting problems. For the fixed σ case, the variance of the noise is constant. However, it helps in optimizing the weights of the network.

4 Experiments

In the experiments, we empirically evaluate ProbAct on image classification and sentiment analysis tasks to show the effectiveness of the proposed method.

Datasets To evaluate our proposed activation function, we use three image classification datasets, CIFAR-10 [36], CIFAR-100 [36], and STL-10 [37] and one text dataset: Large Movie Review [38]. More information on the dataset distribution is mentioned in the Appendix.

4.1 Experimental Setup

To evaluate the performance of the proposed method on classification tasks, we compare ProbAct to the following activation functions: ReLU, Sigmoid, Hyperbolic Tangent (TanH), Leaky ReLU [5], PReLU [6], ELU [39], SELU [40], Swish [9] and to a Bayesian VGG network using variational inference [41]. We utilize a 16-layer VGG neural network [42] architecture for the image classification task and a two-layer CNN network for sentiment analysis task. The architecture, specific hyperparameters, and training settings are provided in the Appendix section.

For a fair and consistent evaluation environment, we did not use regularization tricks, pre-training, and data augmentation to show the true comparison of the activations. The inputs are normalized to $[0, 1]$. The STL-10 images are resized to 32 by 32 to match the CIFAR datasets to keep a fixed input shape to the network.

²Results taken from this implementation :
<https://github.com/kumar-shridhar/PyTorch-BayesianCNN>

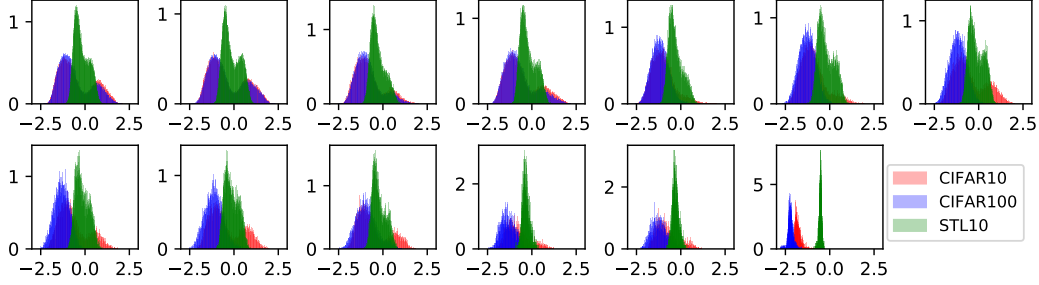


Figure 2: Histograms show how element-wise learnable parameters, k are distributed after training VGG-16 architecture for CIFAR-10, CIFAR-100 and STL-10 datasets at each layer. X-axis denotes k value after training and Y-axis denotes its frequency. Every subfigure represents a ProbAct layer in layer-wise order from top left to bottom right.

For trainable mean, we keep $\sigma = 1$ to see the effects of the learned μ . For our experiments, we train μ element-wise with an initialization of $\mu(x) = \max(0, x)$. Initializing mean with $\max(0, x)$ showed faster convergence when compared with zero or random mean initialization.

For evaluations of σ , we keep $\mu(x) = \max(0, x)$ fixed and we report fixed σ values of 0.5 and 1.0. In case of trainable σ , we set three types of σ values: One Trainable σ , Element-wise (EW) Trainable σ (unbound), and Element-wise Trainable σ (bound). Element-wise Trainable σ (bound) is the Element-wise Trainable σ when σ is bound by $\sigma = \alpha \text{sigmoid}(\beta k)$ and Element-wise Trainable σ (unbound) lacks this constraint. We used $\alpha = 2$ and $\beta = 5$ in the experiments. These values were found through exploratory testing.

We did not see any improvements while training both μ and σ together as their individual accuracy is similar and merging the two modalities did not help. However, training them individually have their own advantages as mentioned in the next section.

4.2 Quantitative Evaluation

The results of the experiment on CIFAR-10, CIFAR-100, STL-10, and IMDB are shown in Table 1. These experiments are performed three times and the average of the three is reported in the results. When using Element-wise Trainable σ (bound) ProbAct, we achieved performance improvements of 2.25% on CIFAR-10, 2.89% on CIFAR-100, 3.37% on STL-10, and 1.5% on IMDB datasets compared to the standard ReLU. In addition, the proposed method performed better than any of the evaluated activation functions.

In order to demonstrate the applicability of our proposed method, the training and testing times relative to the standard ReLU are also shown in Table 1. The time comparison shows that we can achieve higher performance with only a relatively small time difference. This is mainly because the learnable σ values are few compared to the learnable weight values in a network. Hence, our approach comes at nearly no additional time cost. This shows ProbAct as a strong replacement over popular activation functions.

We visualized training aspects of the Single Trainable σ in Figure 3 (a) for 200 epochs for CIFAR-10 dataset. We trained the network for 400 epochs and cropped it for 200 epochs for better visualization as there is no significant change in the σ value after 200 epochs. After 100 training epochs, the Single Trainable σ goes towards 0. However, in order to test the Single Trainable σ when $\sigma = 0$, we replaced ProbAct with ReLU on the trained network. We confirmed that even with ReLU on the network trained with Single Trainable σ , we could achieve higher results than when training on ReLU. This shows that while training, σ helps to optimize the other learnable weights better than standard ReLU architecture, allowing better model performance. Figure 3 (b) shows the mean Element-wise Trainable σ over 200 epochs for all the layers. We demonstrate the ability of the network to train element-wise σ across all layers, even when the number of trainable parameters is increased due to Element-wise Trainable σ parameters.